

PANDUAN PENYUSUNAN & DEPLOYMENT

Implementasi Aplikasi Laravel 11 Berbasis Docker ke Render Cloud

Sistem Target	SiDispen (Sistem Dispensasi Tanggal Dokumen Sekolah Digital)	10 Mei 2026
Arsitektur Lingkungan	Production (Docker Local-to- Cloud Runtime) Database Terkait	PostgreSQL (Supabase Center Location)

Render merupakan platform modern cloud service yang sangat andal untuk menghosting aplikasi web. Secara bawaan (native), Render tidak menyediakan runtime PHP bawaan siap pakai seperti platform lama (Heroku). Oleh karena itu, pendekatan terbaik, paling stabil, dan berstandar industri untuk mendeploy aplikasi Laravel 11 ke Render adalah dengan menggunakan **Docker Container Runtime**.

Penggunaan Docker menjamin seluruh dependensi sistem, versi PHP (8.3+), komponen server Nginx, serta ekstensi database penting seperti `pdo_pgsql` dan `pgsql` terpaket secara utuh dan terhindar dari kendala *"could not find driver"* di lingkungan produksi.

1. Prasyarat Sistem & Persiapan Awal

Sebelum memulai konfigurasi, pastikan poin-poin berikut telah terpenuhi:

- Aplikasi Laravel 11 sudah berjalan normal di lingkungan lokal (Laragon/Localhost).
- Akun aktif di platform **Render.com** dan akun database terkonfigurasi di **Supabase** (PostgreSQL).
- Proyek kodingan sudah diunggah ke repositori Git privat (GitHub atau GitLab).
- Token dan kredensial akses Supabase Storage sudah didapatkan.

2. Pembuatan Struktur File Dockerisasi

Kita akan membuat konfigurasi *Multi-Stage Build* Docker untuk menyatukan PHP-FPM dan Nginx ke dalam satu kontainer terpadu yang efisien, aman, dan ringan (berbasis Alpine Linux). Buatlah direktori baru bernama `docker/` di root proyek Laravel Anda untuk menyimpan file pendukung.

A. File Konfigurasi Nginx (`docker/nginx.conf`)

Render meroutingkan trafik web melalui variabel environment dinamis bernama `PORT`. Oleh karena itu, kita akan menggunakan penanda khusus `LISTEN_PORT` yang nantinya diisi secara otomatis oleh skrip entrypoint kontainer.

```

worker_processes auto;
pid /run/nginx.pid;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
}

http {
    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    access_log /dev/stdout;
    error_log /dev/stderr;

    server {
        listen LISTEN_PORT;
        server_name _;
        root /var/www/html/public;

        add_header X-Frame-Options "SAMEORIGIN";
        add_header X-Content-Type-Options "nosniff";

        index index.php;
        charset utf-8;

        location / {
            try_files $uri $uri/ /index.php?$query_string;
        }

        location = /favicon.ico { access_log off; log_not_found off; }
        location = /robots.txt  { access_log off; log_not_found off; }

        error_page 404 /index.php;

        location ~ \.php$ {
            fastcgi_pass 127.0.0.1:9000;
            fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
            include fastcgi_params;
            fastcgi_hide_header X-Powered-By;
        }

        location ~ /\.(!well-known).* {
            deny all;
        }
    }
}

```

```
}  
}
```

Catatan Penting Konfigurasi Port

Jangan menulis angka 8000 atau 80 secara hardcoded pada perintah `listen` di file Nginx. Render mengalokasikan port secara acak (biasanya port 10000) pada setiap instansi container baru. Penulisan port statis akan menyebabkan container gagal lulus uji *Health Check* Render.

B. Skrip Eksekusi Booting (`docker/entrypoint.sh`)

Skrip ini bertindak sebagai komando awal saat container dinyalakan di Render. Tugasnya meliputi penggantian port dinamis Nginx, pembersihan cache, optimasi aplikasi, running database migration, dan menjalankan PHP-FPM bersama Nginx.

```
#!/bin/sh  
  
# 1. Substitusi variabel PORT dinamis dari Render ke dalam konfigurasi Nginx  
if [ -z "$PORT" ]; then  
    export PORT=80  
fi  
sed -i "s/LISTEN_PORT/$PORT/g" /etc/nginx/nginx.conf  
  
# 2. Jalankan optimalisasi cache Laravel untuk production environment  
echo "Mengoptimalkan konfigurasi Laravel..."  
php artisan config:cache  
php artisan route:cache  
php artisan view:cache  
  
# 3. Jalankan migrasi database Supabase secara otomatis dan aman (force)  
echo "Menjalankan migrasi database database..."  
php artisan migrate --force  
  
# 4. Jalankan PHP-FPM di background mode  
echo "Memulai PHP-FPM 8.3..."  
php-fpm -D  
  
# 5. Jalankan Nginx sebagai foreground process utama container  
echo "Memulai Web Server Nginx pada PORT $PORT..."  
exec nginx -g "daemon off;"
```

C. File Utama Dockerfile (`Dockerfile`)

Buat file bernama `Dockerfile` (tanpa ekstensi) tepat di root direktori proyek Laravel Anda.

```
FROM php:8.3-fpm-alpine
```

```

# 1. Instalasi ekstensi sistem operasi dan driver PostgreSQL (Supabase)
RUN apk add --no-cache      nginx      curl      libpng-dev      libxml2-dev      zip
unzip      git      postgresql-dev

# 2. Kompilasi ekstensi inti internal PHP bawaan yang dibutuhkan Laravel
RUN docker-php-ext-install pdo pdo_pgsql bcmath gd opcache

# 3. Salin konfigurasi kustom Nginx yang telah dibuat sebelumnya
COPY docker/nginx.conf /etc/nginx/nginx.conf

# 4. Tetapkan direktori kerja utama kontainer
WORKDIR /var/www/html

# 5. Salin seluruh codebase aplikasi Laravel ke dalam kontainer
COPY . .

# 6. Ambil binary Composer resmi versi terbaru untuk instalasi dependensi backend
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
ENV COMPOSER_ALLOW_SUPERUSER=1
RUN composer install --no-dev --optimize-autoloader --no-interaction

# 7. Sesuaikan hak akses kepemilikan folder log dan cache agar bisa ditulis oleh Nginx
RUN chown -R www-data:www-data /var/www/html/storage /var/www/html/bootstrap/cache
RUN chmod -R 775 /var/www/html/storage /var/www/html/bootstrap/cache

# 8. Konfigurasi skrip entrypoint operasional booting
COPY docker/entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

# 9. Tetapkan instruksi pembuka kontainer
ENTRYPOINT ["entrypoint.sh"]

```

3. Sinkronisasi Git Repositori

Pastikan Anda mendaftarkan file kustom baru ini ke sistem version control Anda sebelum melakukan deploy:

```

git add docker/nginx.conf docker/entrypoint.sh Dockerfile
git commit -m "Chore: Menambahkan konfigurasi arsitektur Docker untuk Render deployment"
git push origin main

```

4. Langkah Konfigurasi Layanan di Dashboard Render

1. Masuk ke halaman **Dashboard Render** lalu klik tombol **New +** dan pilih opsi **Web Service**.
2. Hubungkan akun Git Anda, lalu pilih repositori proyek Laravel yang ingin disebar.

3. Di lembar konfigurasi awal, atur parameter wajib berikut:

- **Name:** `sidispn-backend-api` (Atau nama preferensi Anda).
- **Region:** Pilih wilayah terdekat dengan server Supabase Anda (direkomendasikan `Singapore`).
- **Branch:** `main` (Atau branch rilis produksi Anda).
- **Runtime:** Wajib pilih opsi **Docker**.
- **Instance Type:** Pilih paket `Free` atau opsi berbayar sesuai beban trafik sekolah.

5. Pengaturan Variabel Lingkungan (Environment Variables)

Buka menu tab **Environment** di dashboard Render Web Service Anda, kemudian masukkan daftar key-value di bawah ini demi menjamin keamanan dan fungsionalitas Laravel 11 Anda:

Key Nama Variabel	Nilai / Value Target	Justifikasi & Deskripsi Fungsional
APP_NAME	SiDispen	Identitas aplikasi resmi sekolah.
APP_ENV	production	Mengunci sistem agar mematikan fitur debugging demi keamanan.
APP_DEBUG	false	Menyembunyikan detail log error sensitif dari layar pengguna luar.
APP_KEY	base64:xxx...xxx	Kunci enkripsi session (Salin nilai persis dari file .env lokal).
DB_CONNECTION	pgsql	Mengaktifkan driver relasional objek database PostgreSQL.
DB_HOST	aws-0-ap-southeast-1...	Alamat endpoint host database yang tertera di panel Supabase.
DB_PORT	5432	Port standar koneksi sistem database PostgreSQL.
DB_DATABASE	postgres	Nama database target operasional utama di Supabase.
DB_USERNAME	postgres	Username kredensial resmi otoritas akses database.
DB_PASSWORD	password_supabase_anda	Kata sandi rahasia akun koneksi database Supabase.
FILESYSTEM_DISK	supabase	Memindahkan disk default dari lokal ke cloud (Supabase Storage).

Peringatan Penting Mengenai Penyimpanan File (Ephemeral Storage)

Arsitektur kontainer server pada Render bersifat **Ephemeral** (sementara). Artinya, setiap kali kontainer melakukan restart atau update kode otomatis, seluruh file fisik yang disimpan di dalam direktori internal kontainer akan **terhapus bersih**. Oleh karena itu, Anda wajib mengonfigurasi penyimpanan eksternal menggunakan driver AWS S3 Flysystem menuju **Supabase Storage Bucket** yang telah dirancang sebelumnya untuk menampung lampiran bukti dispensasi siswa dan lampiran pesan obrolan.

6. Penanganan Masalah & Troubleshooting Umum

A. Error Status 502 Bad Gateway

Penyebab: Biasanya dipicu karena proses PHP-FPM mati mendadak, atau Nginx gagal mendengarkan port eksternal yang dioperasikan dinamis oleh Render.

Solusi: Periksa log di panel dashboard Render. Pastikan skrip `docker/entrypoint.sh` telah diberikan hak eksekusi penuh (`chmod +x`) sebelum di-push ke Git, dan pastikan perintah penggantian string port via `sed` berjalan dengan sukses.

B. Error Koneksi Database: "could not find driver"

Penyebab: Dockerfile Anda lupa mengompilasi library driver pdo_pgsql.

Solusi: Pastikan Dockerfile Anda menggunakan baris kode `RUN docker-php-ext-install pdo pdo_pgsql` dan mengunduh paket system dependency `postgresql-dev` melalui paket manager Alpine OS (apk).

C. Masalah Batas Waktu Uji Kesehatan (Health Check Timeout)

Penyebab: Laravel membutuhkan waktu lama saat menjalankan perintah migrasi database awal yang berat ketika server kontainer pertama kali dipicu, melewati batas tenggat deteksi Render.

Solusi: Buka tab menu **Settings** di dashboard Render Web Service Anda, gulir ke bawah ke arah bagian **Health Check Path**, tetapkan rute ringkas khusus untuk testing ping seperti `/api` atau buat endpoint rute statis minimalis di file `routes/web.php` yang mengembalikan respons string kosong berstatus 200 OK secara instan.